

Introduction to Agile Methodology

Agile Methodology is a modern approach to software development that emphasizes flexibility, customer satisfaction, and rapid delivery. It focuses on developing software in small iterations with continuous feedback and improvement.

Agile Methodology: Concept and Need

Concept of Agile:

Agile follows an iterative and incremental development approach. Software is developed in small parts and delivered frequently.

Customer feedback is taken continuously, and changes are welcomed at any stage of development.

Need for Agile Methodology:

Agile is needed to handle frequent changes in requirements, improve customer satisfaction, reduce risks, and deliver quality software faster.

Agile Manifesto:

The Agile Manifesto was introduced in 2001 and defines the core values and principles of Agile development.

Four Values of Agile Manifesto:

1. Individuals and interactions over processes and tools
2. Working software over comprehensive documentation
3. Customer collaboration over contract negotiation
4. Responding to change over following a plan

Agile Principles:

Agile is based on 12 principles including customer satisfaction, welcoming change, frequent delivery, collaboration, motivated teams, simplicity, technical excellence, self-organizing teams, and continuous improvement.

Scrum Framework:

Scrum is a popular Agile framework used for managing complex projects.

Scrum Roles:

- Product Owner
- Scrum Master
- Development Team

Scrum Artifacts:

- Product Backlog
- Sprint Backlog
- Increment

Scrum Events:

- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

Self-Organizing Teams:

Self-organizing teams decide how to perform their work. They promote collaboration, responsibility, innovation, and high productivity.

Conclusion:

Agile Methodology helps in developing high-quality software through flexibility, customer involvement, and continuous improvement.

Extreme Programming (XP)

Extreme Programming (XP) is an **Agile software development methodology** that focuses on **customer satisfaction**, **high-quality software**, and **frequent delivery** by using simple design and continuous feedback.

Definition

Extreme Programming is an Agile model that improves software quality and responsiveness to changing customer requirements through **short development cycles**, **continuous testing**, and **close customer involvement**.

Core Values of XP

XP is based on **five core values**:

1. **Communication** – Clear and continuous communication among team members and customers
2. **Simplicity** – Develop only what is required, no extra work
3. **Feedback** – Regular feedback from customers and tests

4. **Courage** – Confidence to make changes and improve code
5. **Respect** – Mutual respect within the development team

Key Practices of XP

- **Pair Programming** – Two developers work together on one system
- **Test-Driven Development (TDD)** – Tests are written before code
- **Continuous Integration** – Code is integrated and tested frequently
- **Small Releases** – Software is delivered in small, frequent versions
- **Refactoring** – Improving code without changing functionality
- **On-site Customer** – Customer is continuously available for feedback

XP Process Phases

1. **Planning** – User stories and release planning
2. **Design** – Simple and clear design
3. **Coding** – Pair programming and standard coding practices
4. **Testing** – Unit testing and acceptance testing
5. **Release** – Frequent delivery of working software

Advantages of Extreme Programming

- Fast development
- High-quality and reliable software
- Customer satisfaction is high
- Easily adapts to changing requirements

Disadvantages of Extreme Programming

- Not suitable for large and complex projects
- Requires experienced developers
- Customer availability is mandatory

Applications / When to Use XP

- Small to medium-sized projects

- Projects with frequently changing requirements
- When customer involvement is possible

★ One-Line Conclusion (Very Important for Exams):

Extreme Programming is an Agile methodology that emphasizes **simplicity, continuous feedback, and customer collaboration** to deliver **high-quality software quickly**.

Scrum Process

1. **Product Backlog**
A prioritized list of all requirements and features.
2. **Sprint Planning**
The team selects items from the product backlog to work on in the current sprint (usually 2–4 weeks).
3. **Sprint (Development Phase)**
Actual design, coding, and testing are done.
A **Daily Scrum** (15-minute meeting) is conducted to track progress.
4. **Sprint Review**
At the end of the sprint, the developed product is demonstrated to stakeholders.
5. **Sprint Retrospective**
The team discusses what went well, what problems occurred, and how to improve in the next sprint.

➡ The cycle then repeats for the next sprint.

Scrum Roles

- **Product Owner** – Defines requirements and sets priorities
- **Scrum Master** – Facilitates the process and removes obstacles
- **Development Team** – Develops, tests, and delivers the product

Scrum Artifacts

- Product Backlog
- Sprint Backlog
- Increment (working product at the end of each sprint)

Advantages of Scrum

- Fast and continuous delivery
- Easy to manage changing requirements
- Early customer feedback
- Strong team collaboration

When to Use Scrum

- When requirements are not clearly defined
- When frequent changes are expected
- For complex and dynamic projects

Agile Manifesto

The **Agile Manifesto** was introduced in **2001** to improve software development by focusing on flexibility, collaboration, and customer satisfaction.

Four Core Values of Agile

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

⇒ This means: while items on the right are important, items on the left are valued more.

12 Principles of Agile Software Development

1. **Customer satisfaction** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development, for the customer's competitive advantage.
3. **Deliver working software frequently**, from a few weeks to a few months, with a preference for shorter timescales.
4. **Business people and developers must work together daily** throughout the project.
5. **Build projects around motivated individuals**, give them the environment and support they need, and trust them to get the job done.
6. **Face-to-face conversation** is the most effective and efficient method of conveying information.

7. **Working software is the primary measure of progress.**
8. **Agile processes promote sustainable development** — teams should maintain a constant pace indefinitely.
9. **Continuous attention to technical excellence and good design** enhances agility.
10. **Simplicity** — the art of maximizing the amount of work not done — is essential.
11. **The best architectures, requirements, and designs emerge from self-organizing teams.**
12. **At regular intervals, the team reflects** on how to become more effective and adjusts its behavior accordingly.